

Instituto Tecnológico de San Juan del Río



Ingeniería en Sistemas Computacionales
Tópicos de Ciberseguridad

Reconstrucción del código fuente (Ghidra)

P R E S E N T A N:

Roberto Joshua García Solís 22590410
Oscar Gabriel Rogelio Jiménez Hernández 22590332

Carrera: Ingeniería en Sistemas Computacionales (ISC)

Periodo: Agosto – Diciembre 2026

1. Introducción y objetivo

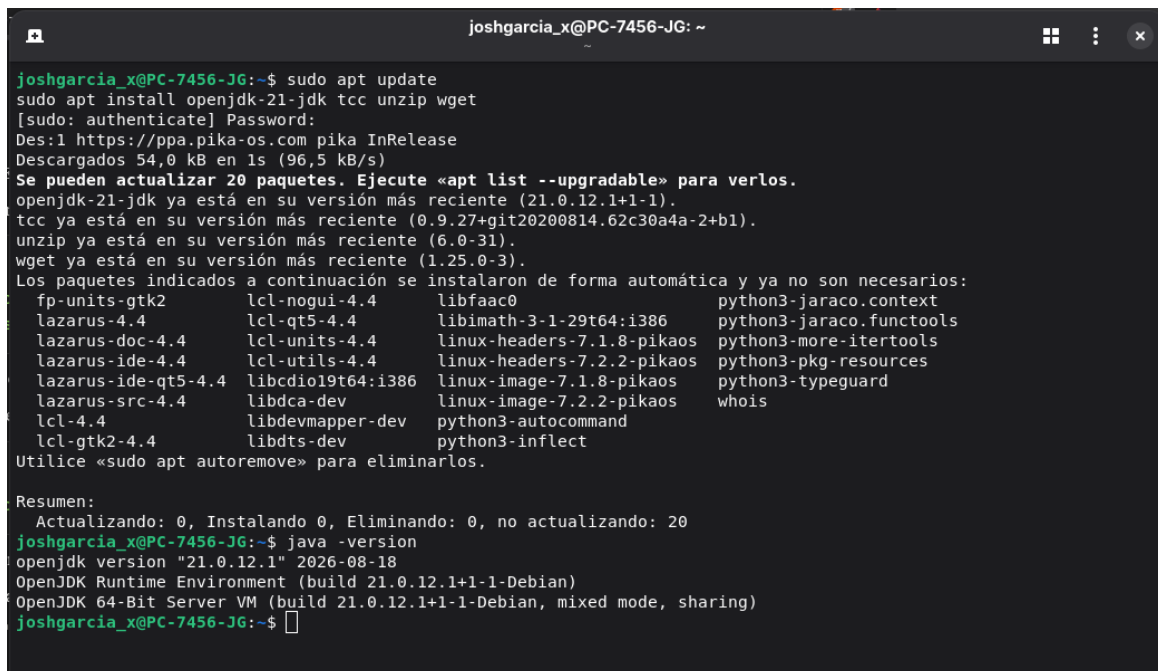
Este documento corresponde al rubro R003 de la actividad "Reconstrucción del código fuente (Ghidra)", de la materia Tópicos de Ciberseguridad. Su objetivo es documentar, paso a paso y con evidencia visual, el proceso seguido para reconstruir el código fuente de un ejecutable compilado con TCC (Tiny C Compiler), utilizando la herramienta Ghidra como analizador y decompilador.

El resultado de este proceso (el código fuente reconstruido) y su verificación mediante recompilación y comparación contra el original se documentan por separado en el rubro R004.

2. Herramientas y entorno de trabajo

- Sistema operativo: PikaOS (basado en Ubuntu/Debian).
- Ghidra 12.1 (build pública, NationalSecurityAgency/ghidra, tag Ghidra_12.1_build).
- TCC (Tiny C Compiler) 0.9.27, instalado desde los repositorios de PikaOS.
- Java (OpenJDK) 21.0.12, requerido para ejecutar Ghidra.

A continuación se documenta la instalación de estas herramientas.



```
joshgarcia_x@PC-7456-JG: ~  
joshgarcia_x@PC-7456-JG:~$ sudo apt update  
sudo apt install openjdk-21-jdk tcc unzip wget  
[sudo: authenticate] Password:  
Des:1 https://ppa.pika-os.com pika InRelease  
Descargados 54,0 kB en 1s (96,5 kB/s)  
Se pueden actualizar 20 paquetes. Ejecute «apt list --upgradable» para verlos.  
openjdk-21-jdk ya está en su versión más reciente (21.0.12.1+1-1).  
tcc ya está en su versión más reciente (0.9.27+git20200814.62c30a4a-2+b1).  
unzip ya está en su versión más reciente (6.0-31).  
wget ya está en su versión más reciente (1.25.0-3).  
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios:  
fp-units-gtk2      lcl-nogui-4.4      libfaac0           python3-jaraco.context  
lazarus-4.4        lcl-qt5-4.4        libimath-3-1-29t64:i386  python3-jaraco.functools  
lazarus-doc-4.4    lcl-units-4.4      linux-headers-7.1.8-pikaos  python3-more-itertools  
lazarus-ide-4.4    lcl-utils-4.4      linux-headers-7.2.2-pikaos  python3-pkg-resources  
lazarus-ide-qt5-4.4  libcdio19t64:i386  linux-image-7.1.8-pikaos  python3-typeguard  
lazarus-src-4.4     libdca-dev         linux-image-7.2.2-pikaos  whois  
lcl-4.4            libdevmapper-dev   python3-autocommand  
lcl-gtk2-4.4       libdts-dev         python3-infect  
Utilice «sudo apt autoremove» para eliminarlos.  
  
Resumen:  
  Actualizando: 0, Instalando 0, Eliminando 0, no actualizando: 20  
joshgarcia_x@PC-7456-JG:~$ java -version  
openjdk version "21.0.12.1" 2026-08-18  
OpenJDK Runtime Environment (build 21.0.12.1+1-1-Debian)  
OpenJDK 64-Bit Server VM (build 21.0.12.1+1-1-Debian, mixed mode, sharing)  
joshgarcia_x@PC-7456-JG:~$
```

Figura 1. Instalación de OpenJDK 21, TCC, unzip y wget mediante apt, y verificación de la versión de Java instalada.

[illegible]

Figura 2. Descarga del paquete de Ghidra 12.1 desde el repositorio oficial de GitHub (NationalSecurityAgency/ghidra).

3. Programa analizado

Se utilizó un programa propio y sencillo, `reporte_temperaturas.c`, que simula un arreglo fijo de 7 lecturas de temperatura (una por día de la semana), calcula estadísticas (mínimo, máximo, promedio y número de días con alerta de calor) y genera un archivo `reporte.json` con el detalle. El programa se compiló con TCC:

```
tcc -o reporte_temperaturas reporte_temperaturas.c -lm
```

El binario resultante es un ejecutable ELF de 5224 bytes, compilado sin símbolos de depuración (stripped), lo cual obliga a un análisis real por parte del decompilador en lugar de una simple lectura de nombres de función. El código fuente completo (original y reconstruido) se entrega en el rubro R004, junto con la comparación.

```
joshgarcia_x@PC-7456-JG:~/Descargas$ ls ~/ghidra
ghidra_12.1_PUBLIC
joshgarcia_x@PC-7456-JG:~/Descargas$ cd /home/joshgarcia_x/Descargas/Reconstruccion_Ghidra_TCC/entrega/original
joshgarcia_x@PC-7456-JG:~/Descargas/Reconstruccion_Ghidra_TCC/entrega/original$ tcc -o reporte_temperaturas reporte_temperaturas.c -lm
joshgarcia_x@PC-7456-JG:~/Descargas/Reconstruccion_Ghidra_TCC/entrega/original$ ./reporte_temperaturas
Generando reporte de temperaturas...
Reporte generado exitosamente en reporte.json
Mínimo: 19.8, Máximo: 33.1, Promedio: 26.81, Días de alerta: 2
joshgarcia_x@PC-7456-JG:~/Descargas/Reconstruccion_Ghidra_TCC/entrega/original$
```

Figura 3. Compilación del programa original con TCC y ejecución, confirmando la salida esperada en consola.

4. Desarrollo — Proceso de reconstrucción

4.1 Creación del proyecto e importación del ejecutable en Ghidra

Se abrió Ghidra (/ghidraRun) y se creó un nuevo proyecto no compartido ("Non-Shared Project") llamado ReconstruccionTemperaturas.

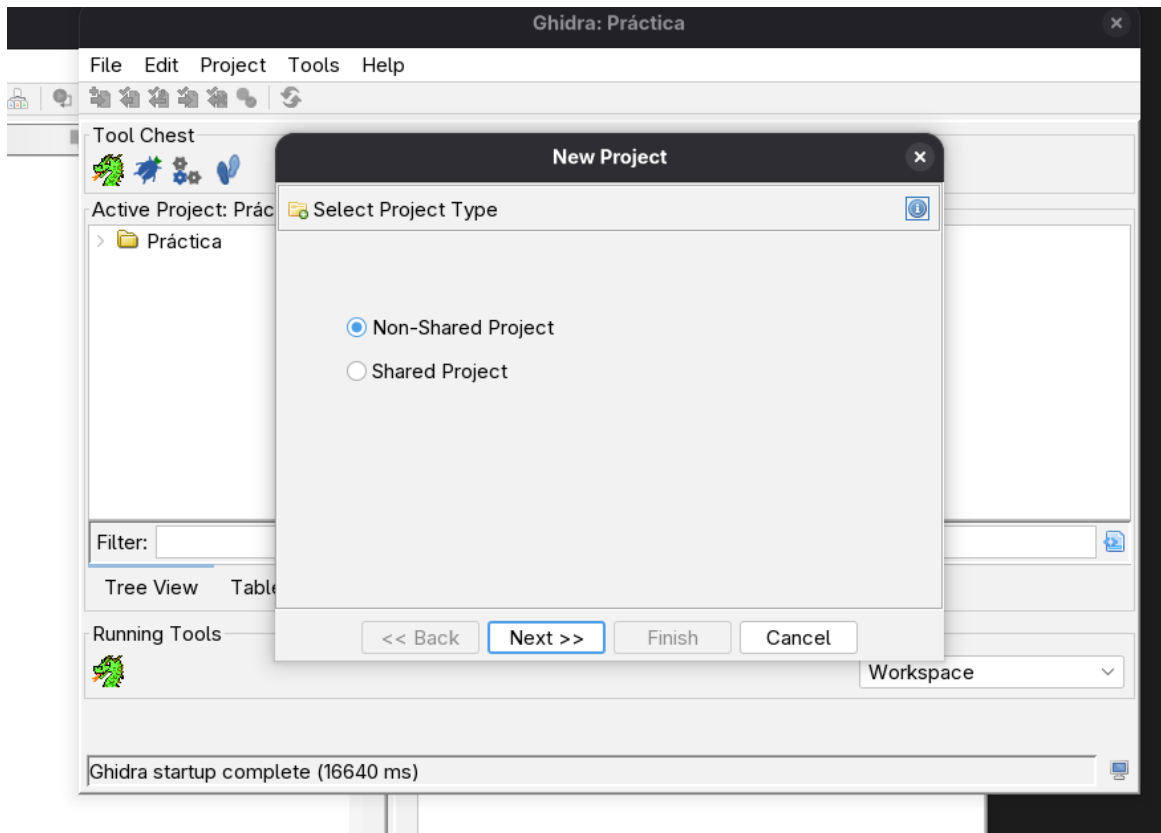


Figura 4. Selección del tipo de proyecto: Non-Shared Project.

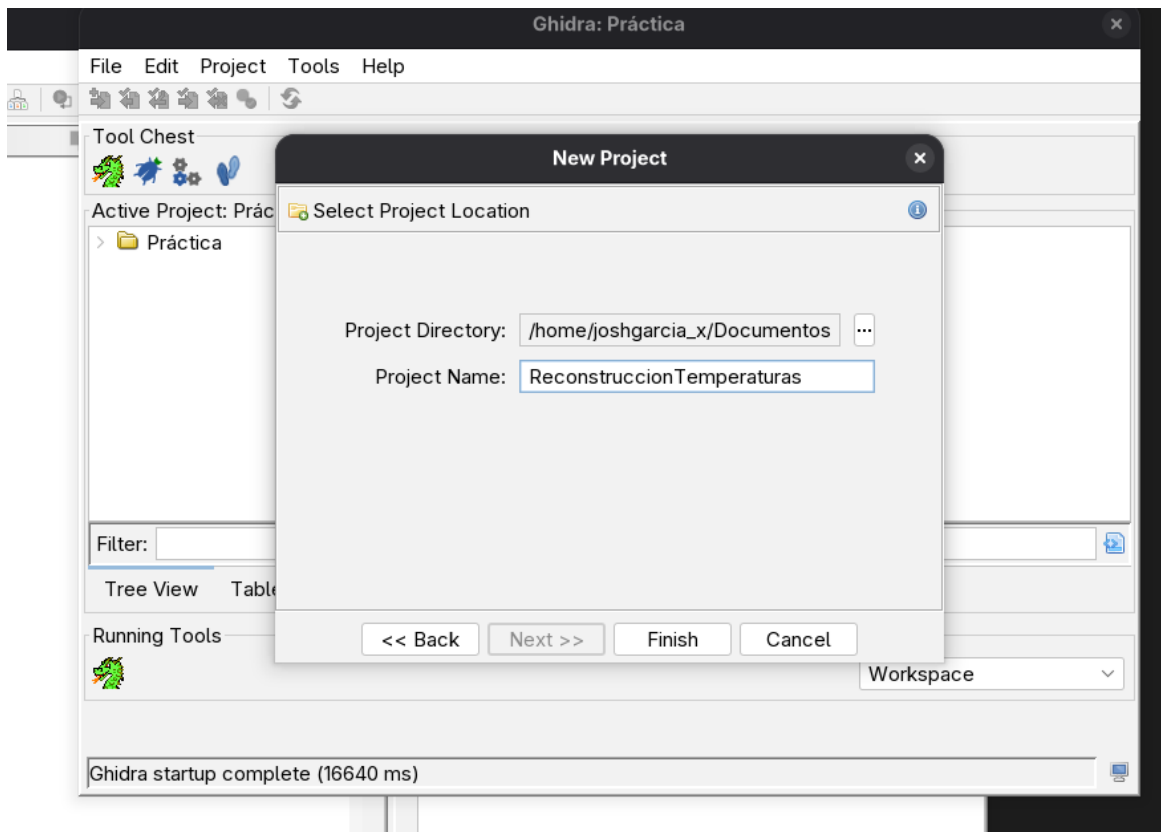


Figura 5. Definición de la ubicación y nombre del proyecto (ReconstruccionTemperaturas).

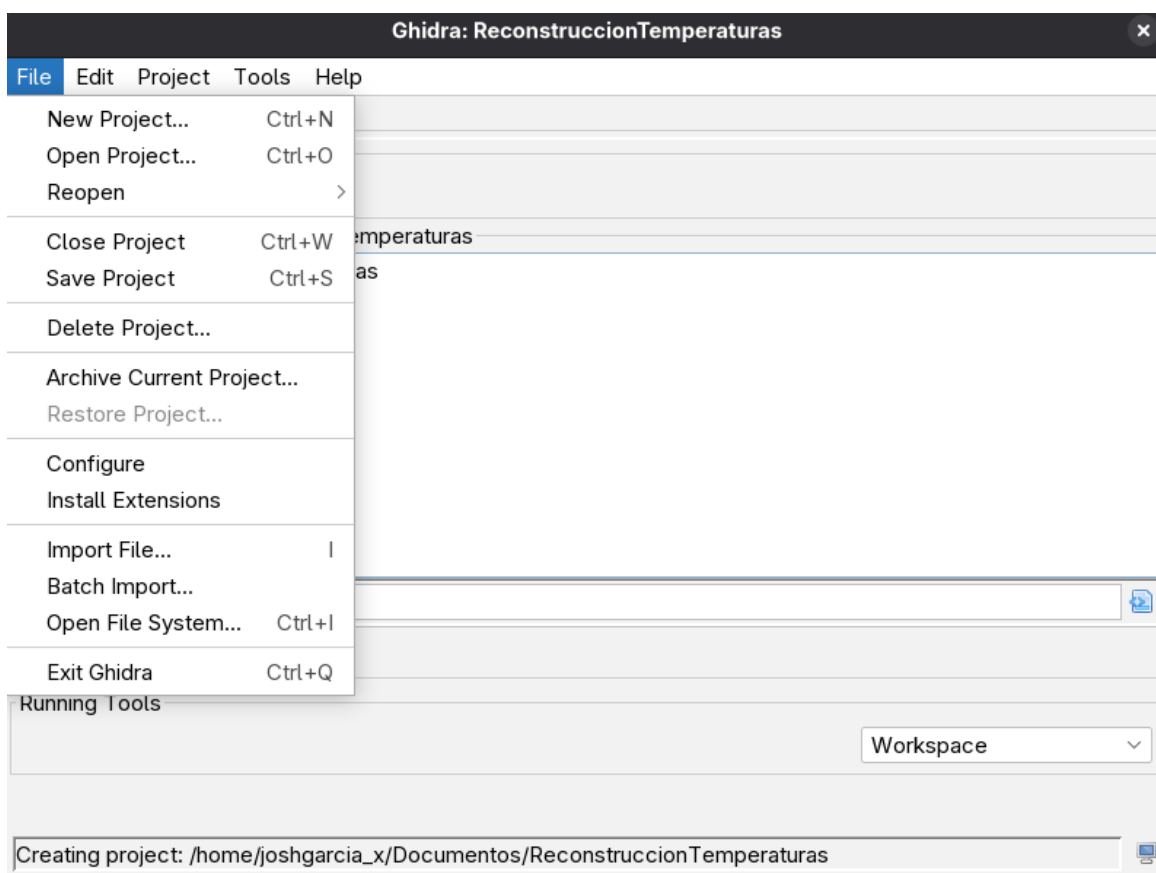


Figura 6. Menú File → Import File... utilizado para importar el ejecutable.

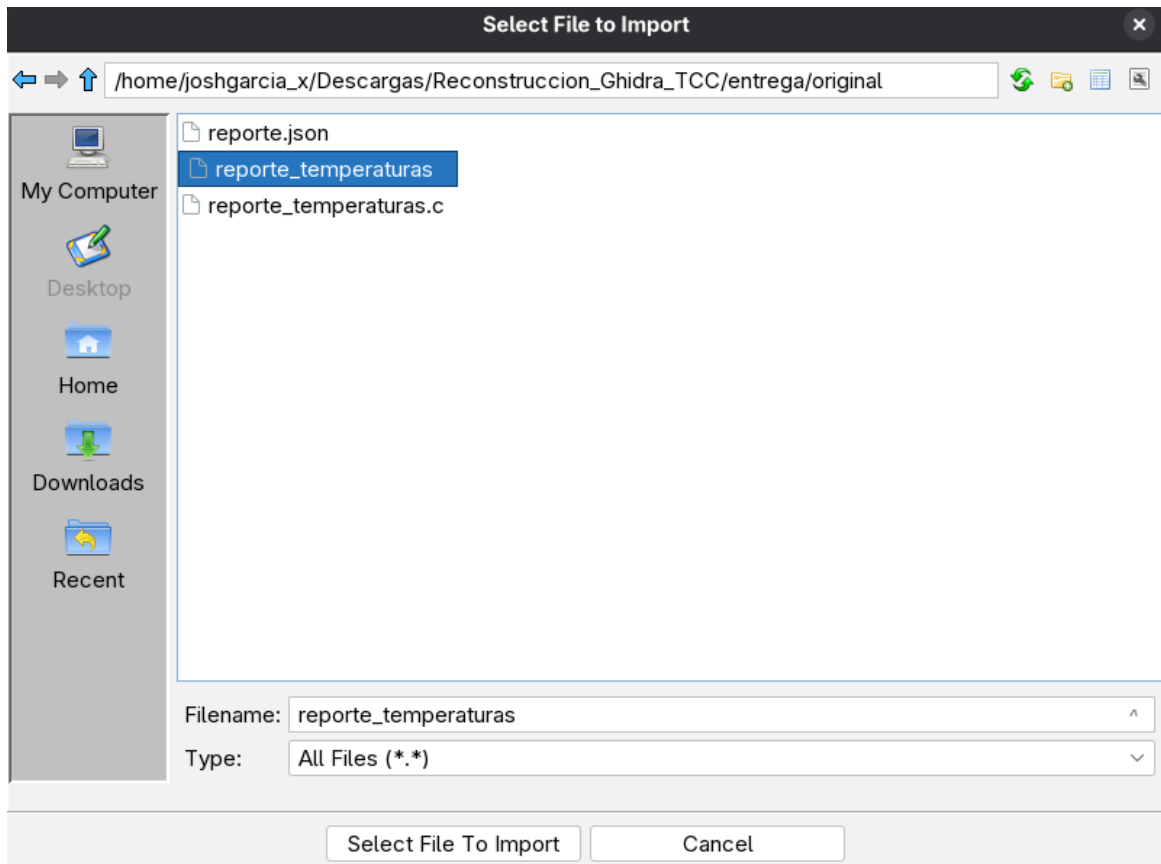


Figura 7. Selección del archivo `reporte_temperaturas` (el ejecutable compilado con TCC) para su importación.

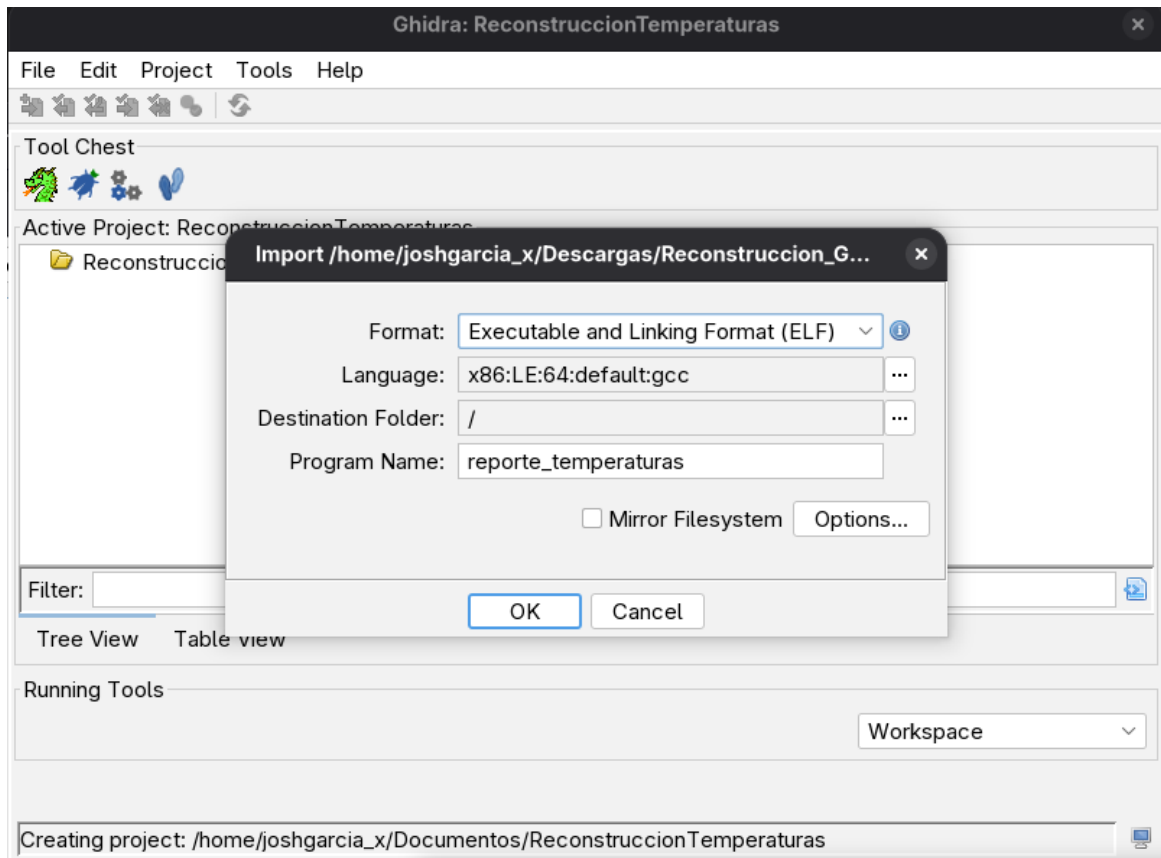


Figura 8. Cuadro de opciones de importación: Ghidra detecta automáticamente el formato "Executable and Linking Format (ELF)" y el lenguaje/arquitectura x86:LE:64:default:gcc.

El resumen de importación confirma metadatos clave del binario: tamaño, número de funciones detectadas preliminarmente, y el hash SHA-256 del ejecutable, útil como huella digital para la comparación posterior.

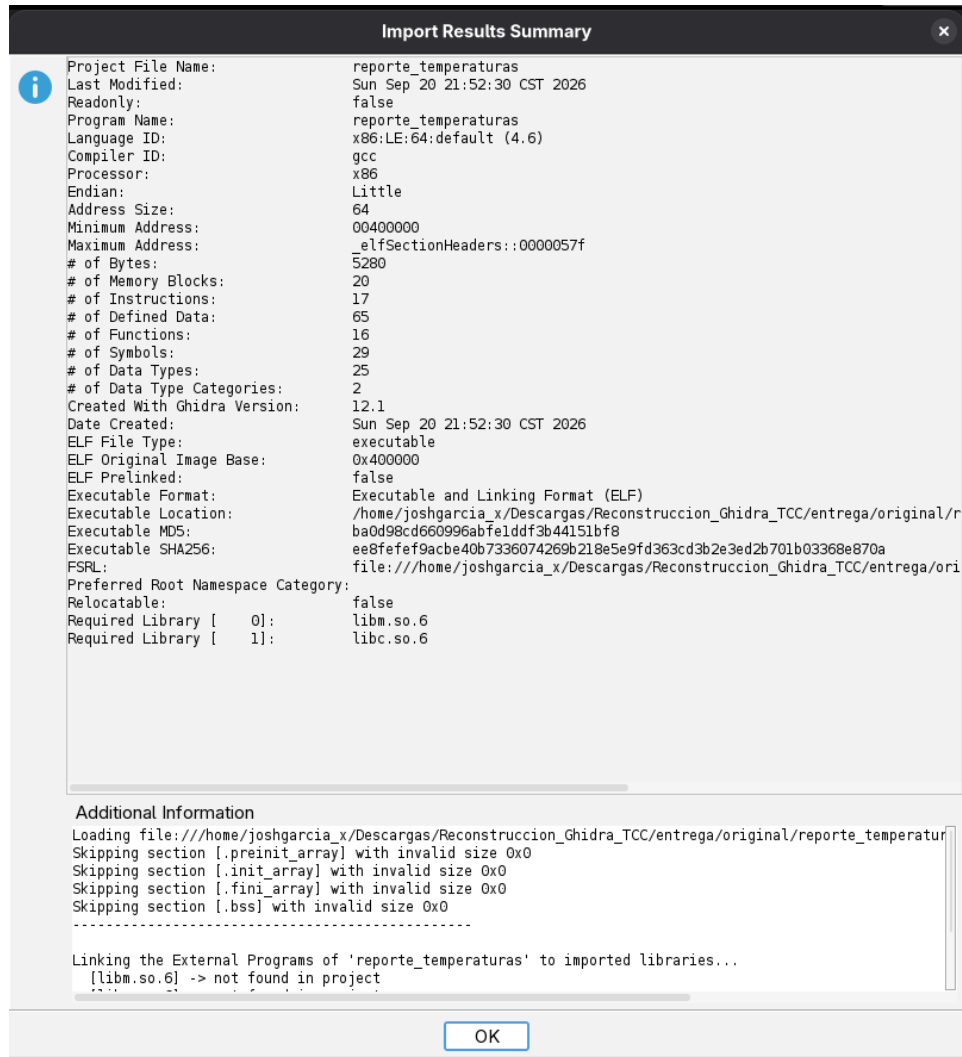


Figura 9. Resumen de importación (Import Results Summary), con el hash SHA-256 del ejecutable, tamaño, arquitectura y librerías requeridas (libc.so.6, libm.so.6).

4.2 Análisis automático

Al abrir el CodeBrowser, Ghidra detectó que el binario no había sido analizado y ofreció ejecutar el análisis automático con los analizadores estándar (ASCII Strings, Decompiler Switch Analysis, Function ID, Stack, etc.), los cuales se dejaron con su configuración por defecto.

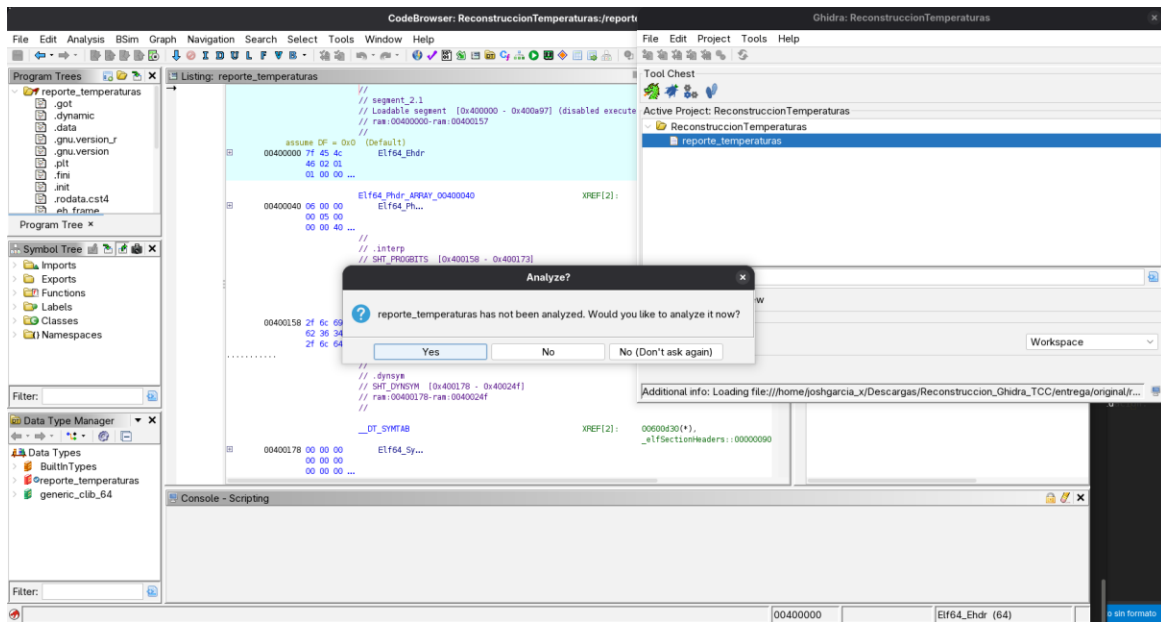


Figura 10. Diálogo "Analyze?" al abrir el ejecutable por primera vez.

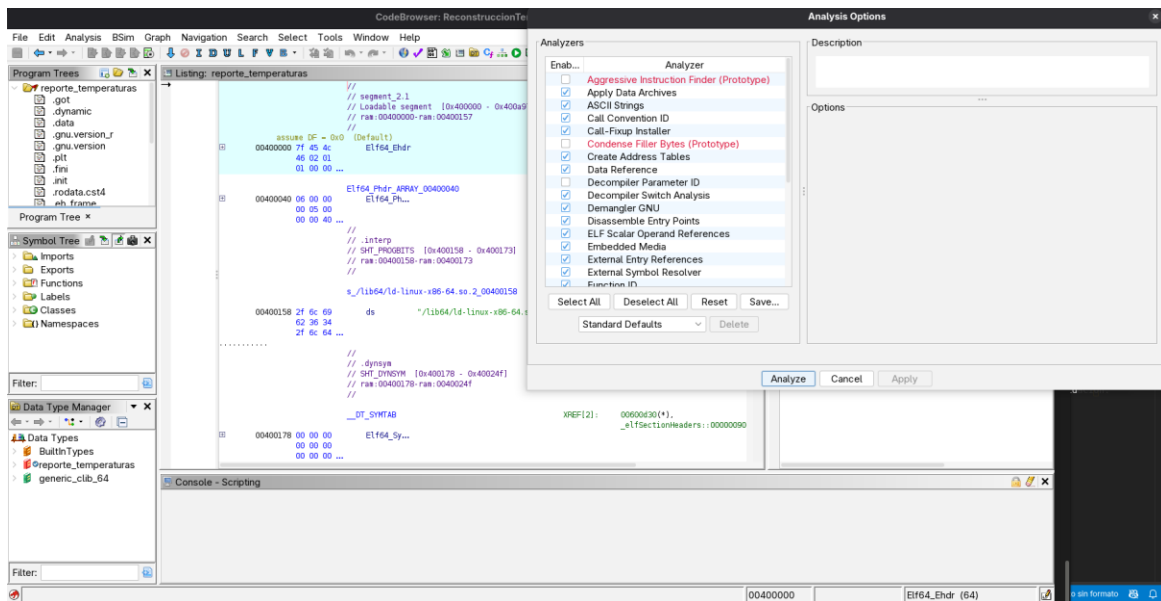


Figura 11. Ventana de Analysis Options, con la lista de analizadores activos por defecto.

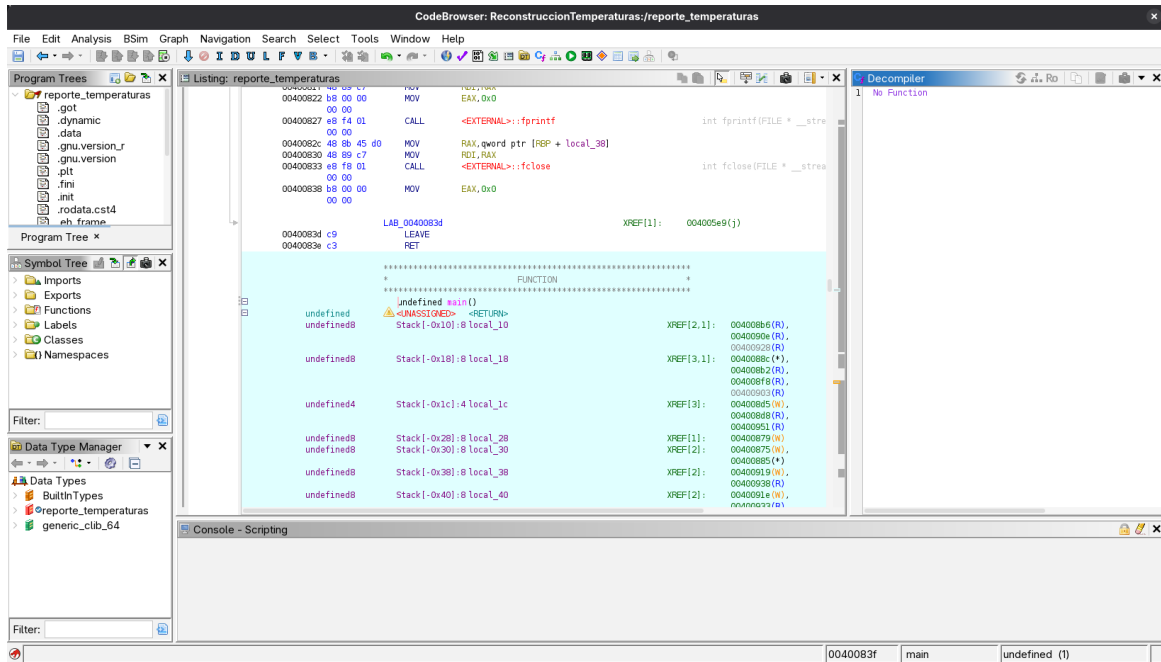


Figura 12. CodeBrowser tras finalizar el análisis automático.

4.3 Exploración del árbol de funciones

En el panel Symbol Tree, dentro de la carpeta Functions, Ghidra identificó 9 funciones en total, incluyendo el punto de entrada (entry), los inicializadores del runtime (_DT_INIT, _DT_FINI) y 4 funciones propias del programa, que Ghidra nombró de forma genérica por no existir símbolos de depuración: FUN_00400400, FUN_00400410, FUN_00400411, FUN_0040045d, FUN_00400582 y FUN_004009f0, además de main.

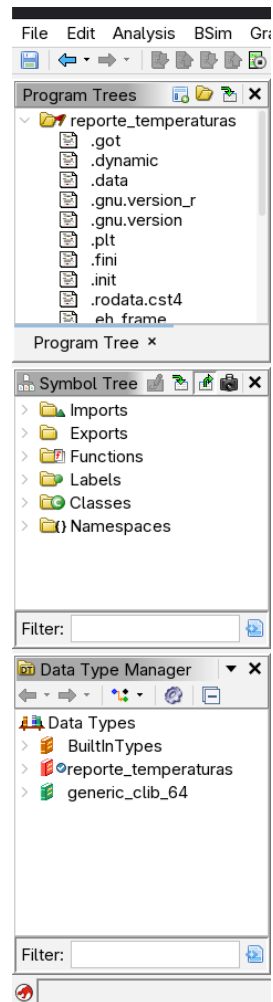


Figura 13. Panel Symbol Tree con la carpeta Functions colapsada.

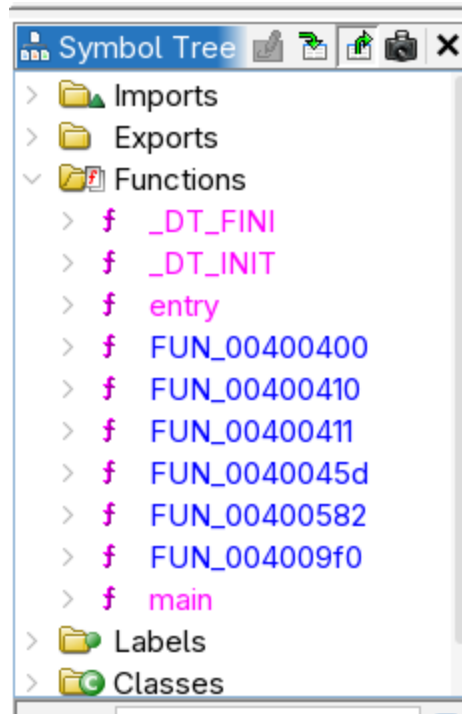


Figura 14. Carpeta Functions expandida: se listan las 9 funciones detectadas por Ghidra.

4.4 Decompilación función por función

Se revisó el panel Decompile de cada función relevante para entender su lógica y renombrarla adecuadamente en la reconstrucción (ver el código final en R004).

a) Función main

Se observa que main llama a una función con un arreglo y el número 7 (los 7 días), que retorna un bloque de 16 bytes (undefined1 [16]). Ese bloque se copia con memmove a 4 variables (mínimo, máximo, promedio, días de alerta), y luego se llama a una segunda función pasándole esos 4 valores más el arreglo original — presumiblemente la que escribe el JSON.

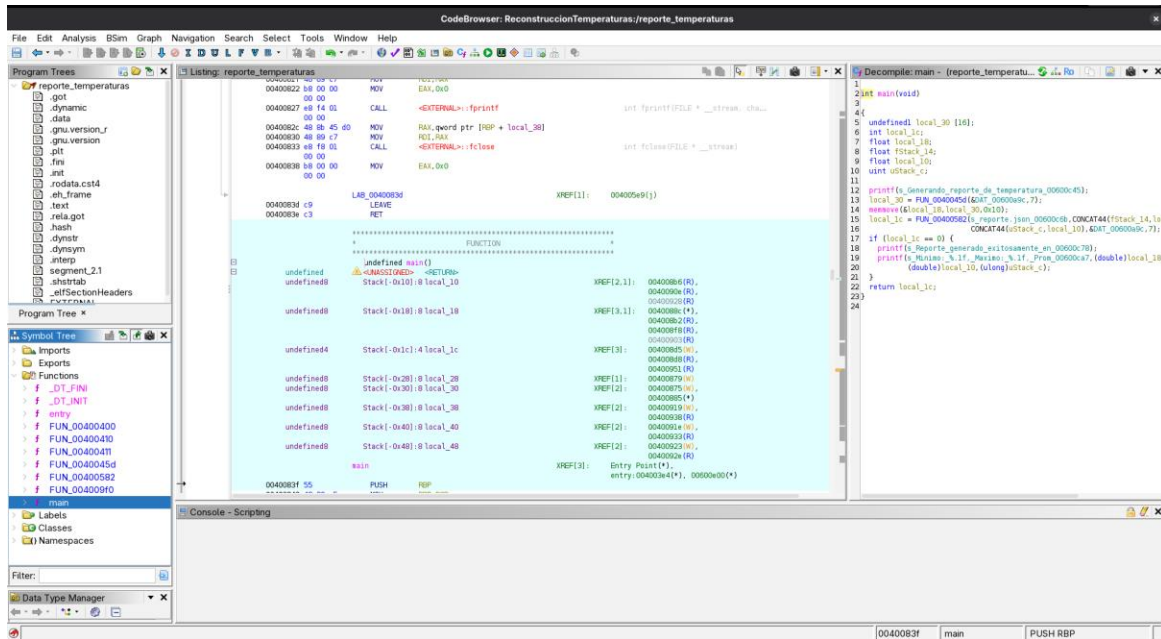


Figura 15. Pseudo-código C de la función main generado por el decompilador de Ghidra.

b) Función de estadísticas (FUN_0040045d)

Recorre un arreglo de floats llevando un mínimo, un máximo, una suma acumulada y un contador que se incrementa cuando el valor supera una constante de referencia. Al final empaqueta los 4 resultados en un bloque de 16 bytes — el mismo patrón observado en main — lo que confirma que corresponde a una estructura de 4 campos: {float minimo; float maximo; float promedio; int dias_alerta;}.

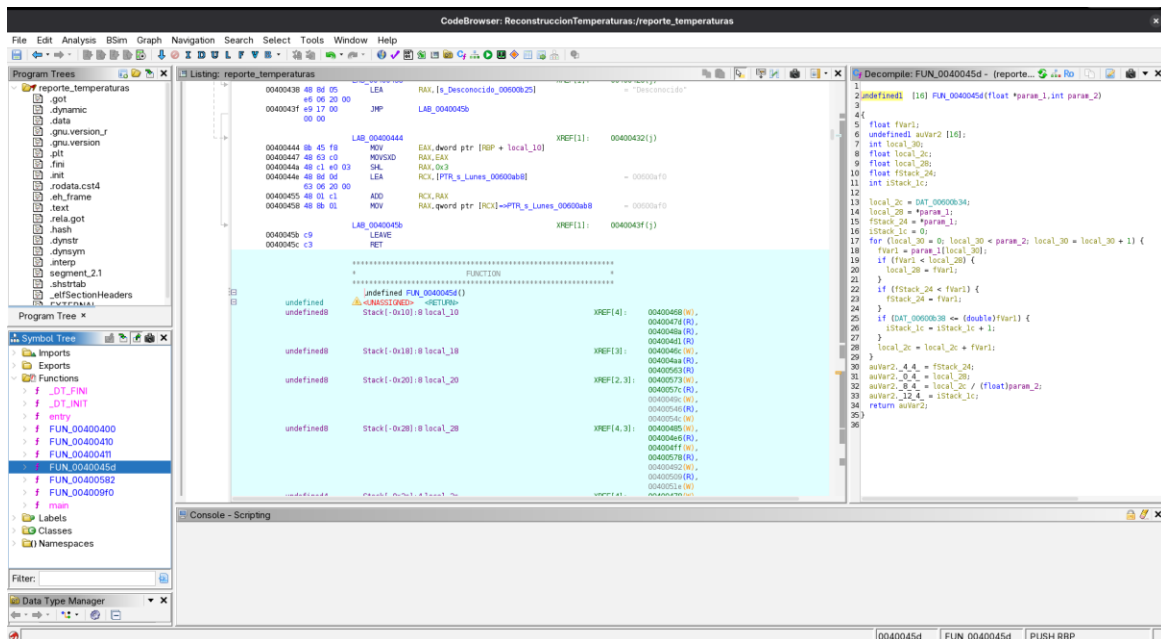


Figura 16. Pseudo-código de la función de cálculo de estadísticas (mínimo, máximo, promedio, días de alerta).

c) Función de nombre del día (FUN_00400411)

Valida que el índice recibido esté en el rango 0–6; si está fuera de rango regresa el string "Desconocido", y en caso contrario indexa un arreglo de punteros a cadenas de texto (los nombres de los días).

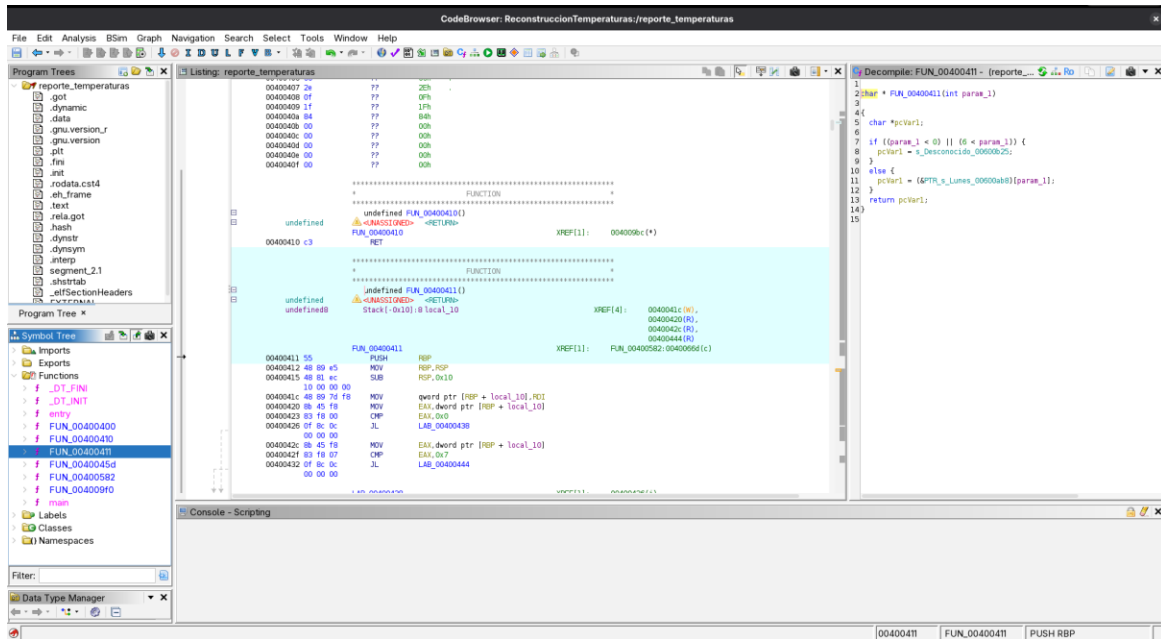


Figura 17. Pseudo-código de la función que traduce un índice numérico al nombre del día correspondiente.

d) Función de escritura del reporte JSON (FUN_00400582)

Abre un archivo con fopen, y mediante una serie de llamadas a fprintf construye el contenido en formato JSON (encabezado, arreglo de lecturas por día, bloque de estadísticas), para finalmente cerrarlo con fclose. Ghidra dejó visibles en el propio pseudo-código varios de los literales de texto usados en los formatos.

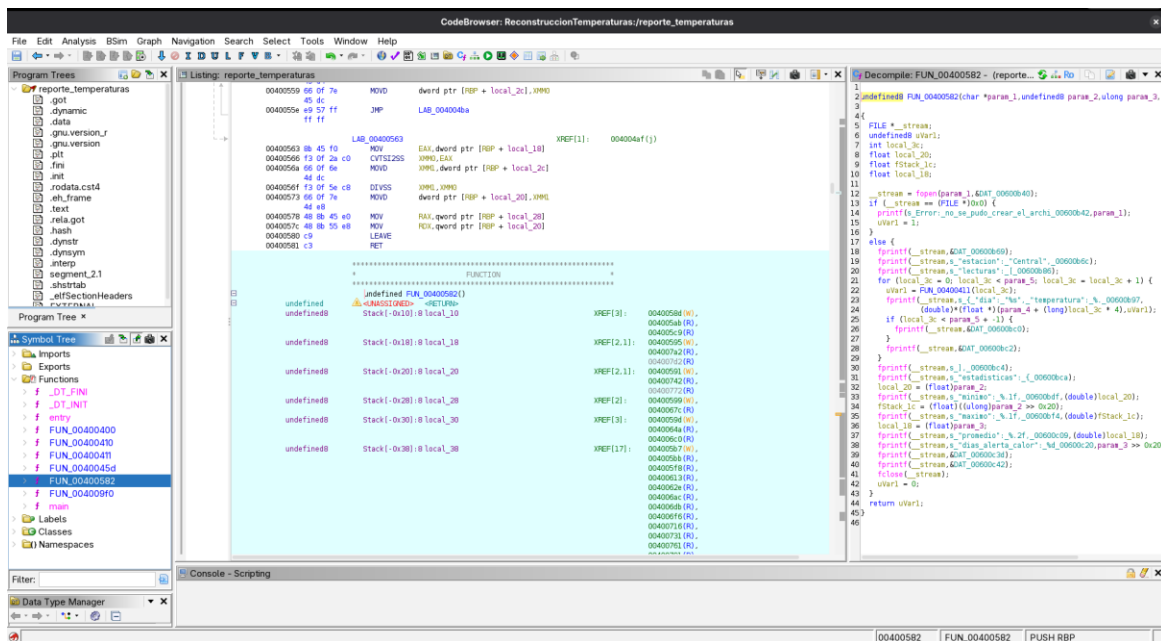


Figura 18. Pseudo-código de la función que construye y escribe el archivo reporte.json.

4.5 Recuperación de cadenas de texto

Para confirmar y completar los literales de texto usados por el programa (más allá de lo que ya se alcanzaba a leer en el pseudo-código), se utilizó la ventana Window → Defined Strings, que lista todas las cadenas presentes en el binario.

Location	String Value	String Representation	Data Type
.shstrtab:00000001	.text	u8".text"	utf8
.shstrtab:00000007	.data	u8".data"	utf8
.shstrtab:0000000d	.bss	u8".bss"	utf8
.shstrtab:00000012	.eh_frame	u8".eh_frame"	utf8
.shstrtab:0000001c	.rodata.cst4	u8".rodata.cst4"	utf8
.shstrtab:00000029	.init	u8".init"	utf8
.shstrtab:0000002f	.fini	u8".fini"	utf8
.shstrtab:00000035	.preinit_array	u8".preinit_array"	utf8
.shstrtab:00000044	.init_array	u8".init_array"	utf8
.shstrtab:00000050	.fini_array	u8".fini_array"	utf8
.shstrtab:0000005c	.interp	u8".interp"	utf8
.shstrtab:00000064	.dynsym	u8".dynsym"	utf8
.shstrtab:0000006c	.dynstr	u8".dynstr"	utf8
.shstrtab:00000074	.hash	u8".hash"	utf8
.shstrtab:0000007a	.dynamic	u8".dynamic"	utf8
.shstrtab:00000083	.got	u8".got"	utf8
.shstrtab:00000088	.rela.got	u8".rela.got"	utf8
.shstrtab:00000092	.plt	u8".plt"	utf8
.shstrtab:00000097	.gnu.version	u8".gnu.version"	utf8
.shstrtab:000000a4	.gnu.version_r	u8".gnu.version_r"	utf8
.shstrtab:000000b3	.shstrtab	u8".shstrtab"	utf8
00400001	ELF	*ELF"	ds
00400158	/lib64/ld-linux-x86-64.so.2	*/lib64/ld-linux-x86-64.so.2"	ds
00400251	_libc_start_main	u8"_libc_start_main"	utf8
00400263	fopen	u8"fopen"	utf8
00400269	printf	u8"printf"	utf8
00400270	fprintf	u8"fprintf"	utf8
00400278	fclose	u8"fclose"	utf8
0040027f	memmove	u8"memmove"	utf8
00400287	main	u8"main"	utf8
0040028c	__gmon_start__	u8"__gmon_start__"	utf8
0040029b	libc.so.6	u8"libc.so.6"	utf8
004002a5	GLIBC_2.34	u8"GLIBC_2.34"	utf8
004002b0	GLIBC_2.2.5	u8"GLIBC_2.2.5"	utf8
004002bc	libm.so.6	u8"libm.so.6"	utf8
004002c6	libc.so.6	u8"libc.so.6"	utf8
00400961	zR	*zR"	ds
00400991	zR	*zR"	ds
00600af0	Lunes	*Lunes"	ds
00600af6	Martes	*Martes"	ds
00600afd	Miercoles	*Miercoles"	ds

Figura 19. Ventana Defined Strings (parte 1): nombres de librerías, símbolos importados y los primeros días de la semana.

00600b07	Jueves	*Jueves"	ds
00600b0e	Viernes	*Viernes"	ds
00600b16	Sabado	*Sabado"	ds
00600b1d	Domingo	*Domingo"	ds
00600b25	Desconocido	*Desconocido"	ds
00600b42	Error: no se pudo crear el archivo %s	*Error: no se pudo crear el archivo %s\n"	ds
00600b6c	"estacion": "Central",	* "estacion": "\Central",\n"	ds
00600b86	"lecturas": {	* "lecturas": {\n"	ds
00600b97	{ "dia": "%s", "temperatura": %.1f }	* { "\dia": "%s", "temperatura": %.1f...	ds
00600bc4	},	* },\n"	ds
00600bca	"estadisticas": {	* "estadisticas": {\n"	ds
00600bdf	"minimo": %.1f,	* "\minimo": %.1f,\n"	ds
00600bf4	"maximo": %.1f,	* "\maximo": %.1f,\n"	ds
00600c09	"promedio": %.2f,	* "\promedio": %.2f,\n"	ds
00600c20	"dias_alerta_calor": %d	* "\dias_alerta_calor": %d\n"	ds
00600c45	Generando reporte de temperaturas...	*Generando reporte de temperaturas...\n"	ds
00600c6b	reporte.json	*reporte.json"	ds
00600c78	Reporte generado exitosamente en repor...	*Reporte generado exitosamente en repo...	ds
00600ca7	Mínimo: %.1f, Máximo: %.1f, Promedio: %...	*Mínimo: %.1f, Máximo: %.1f, Promedio: ...	ds

Figura 20. Ventana Defined Strings (parte 2): el resto de los días, el mensaje "Desconocido", y los formatos completos del JSON y de los mensajes de consola.

4.6 Recuperación de las constantes numéricas

Los 7 valores de temperatura no aparecen como texto (son números binarios tipo float), por lo que se recuperaron navegando directamente a la dirección de memoria referenciada en main (DAT_00600a9c) usando Navigation → Go To.

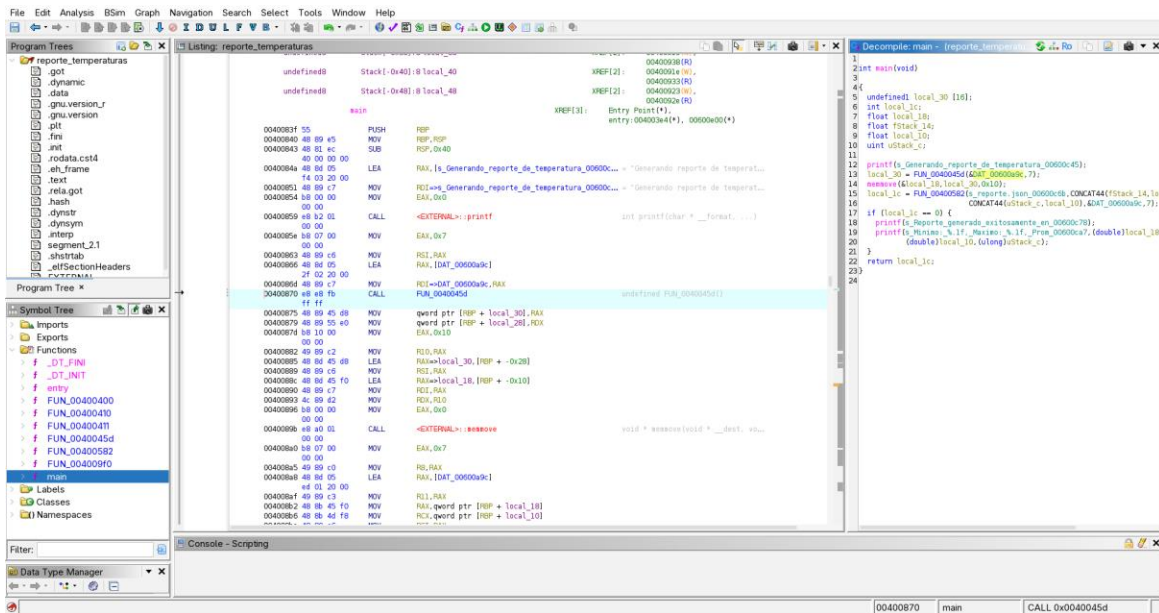


Figura 21. Función main con la referencia a DAT_00600a9c resaltada, punto de partida para localizar el arreglo de temperaturas.

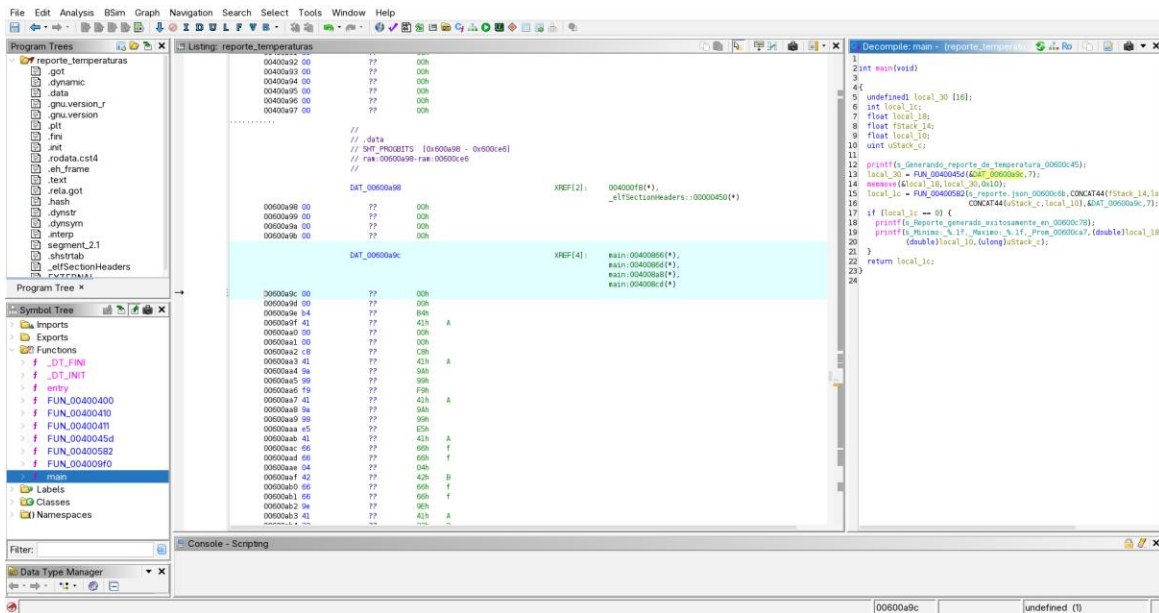


Figura 22. Bytes crudos en el panel Listing, en la dirección 00600a9c, antes de interpretarlos como números.

Para leer estos bytes como números decimales, se seleccionó el primer bloque de 4 bytes y se aplicó, mediante clic derecho, la opción Data → float, lo que reveló el primer valor: 22.5.

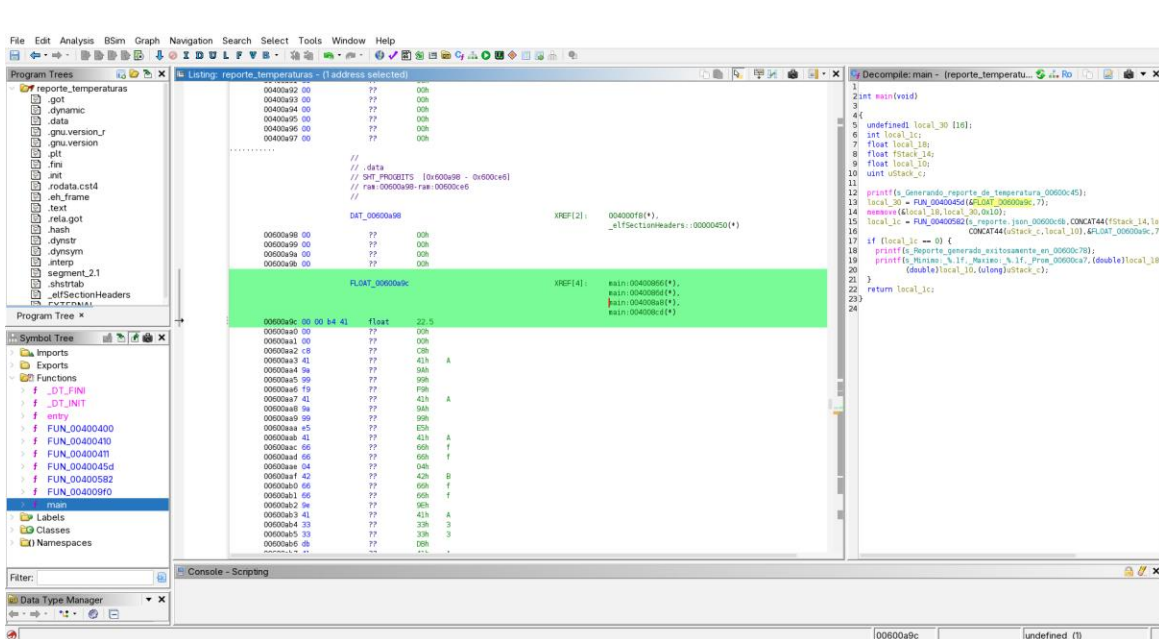


Figura 23. Primer valor convertido a tipo float por Ghidra: 22.5, en la dirección 00600a9c.

Repetiendo la conversión en los siguientes bloques de 4 bytes se obtuvieron los 7 valores completos del arreglo de temperaturas: 22.5, 25.0, 31.2, 28.7, 33.1, 19.8 y 27.4. Adicionalmente, justo debajo se pudo observar el arreglo de punteros a cadenas (PTR_s_Lunes_00600ab8) que confirma la estructura usada por la función que traduce el índice al nombre del día.

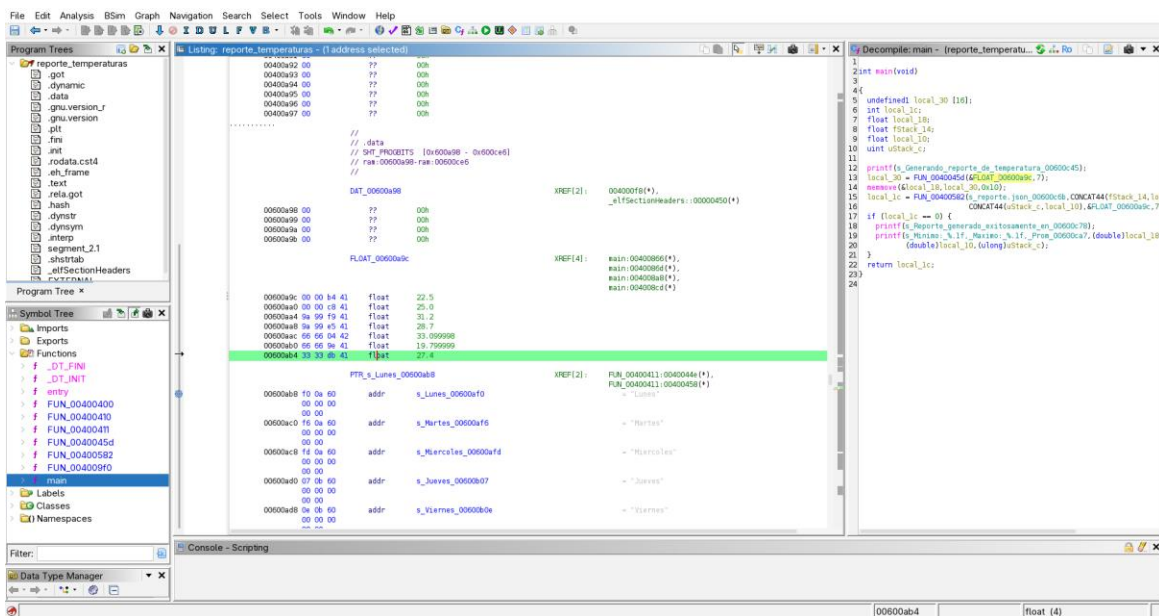


Figura 24. Los 7 valores de temperatura recuperados en formato decimal (float), junto con el arreglo de punteros a los nombres de los días.

5. Conclusión del proceso

Con la lógica de las 4 funciones propias del programa, las cadenas de texto recuperadas de la ventana Defined Strings y las 7 constantes numéricas obtenidas del volcado de memoria, se reunió toda la información necesaria para

reescribir el código fuente en C. La reconstrucción del código, su recompilación con TCC y la comparación contra el ejecutable original se documentan en el rubro R004.

Referencias

- Ghidra Software Reverse Engineering Framework — National Security Agency.
<https://github.com/NationalSecurityAgency/ghidra>
- Documentación oficial de Ghidra (analyzeHeadlessREADME, Decompiler, Defined Strings).